# Lesson 9 — Assignment Topics: Consensus, Multi-Agent Systems, Distributed Search, P2P, MapReduce

**INTE 22253 | Source: Assignment 01 Part A and Part C, plus the module outline in Lecture 01 (slides 10–11). These topics have NO lecture slides of their own, so this lesson is written from standard textbooks (Coulouris; Tanenbaum & van Steen; the Raft, Chord and MapReduce papers).**

**HOW TO USE THIS LESSON**

The final exam is most likely to follow the lectures (Lessons 1–8). Study this lesson **after** those, so you can answer questions that touch the assignment themes (your viva and report also needed them). Everything here is standard theory and does not contradict the slides: 2PC/3PC come from Lecture 04; MapReduce, GFS/HDFS/S3 from Lectures 01 and 07.

**WHAT THE ASSIGNMENT ASKED (PART A, 20 MARKS; PART C, 30 MARKS)**

1. **Distributed transactions:** explain 2PC, its weaknesses and how 3PC addresses them; how consensus protocols (Paxos/Raft) support coordination; why consensus is harder than transaction commitment.
2. **Multi-agent systems:** key characteristics; how agents coordinate, communicate and resolve conflicts over shared resources.
3. **Distributed search engines:** indexing and retrieval across nodes (inverted index, sharding); centralized vs distributed/P2P search (scalability, fault tolerance, latency).
4. **P2P systems:** structured (Chord, Kademlia, DHTs) vs unstructured (Gnutella flooding): lookup efficiency, churn tolerance, complexity.
5. **Part C:** design a DFS for 1–10 GB log files (Lesson 7) *or* implement MapReduce word count / an S3 client and explain the cloud programming model (MapReduce, serverless/FaaS, actor model).

## 1. From commit to consensus

### 1.1 Recap: 2PC and 3PC (Lesson 4)

2PC: coordinator sends VOTE-REQUEST; all YES ⇒ COMMIT, any NO ⇒ ABORT. **Weaknesses:** (1) **blocking** if the coordinator crashes after YES votes; (2) the coordinator is a **single point of failure**; (3) participants **hold locks** while waiting, hurting throughput; (4) many messages and log writes (latency). 3PC adds PREPARE-TO-COMMIT so survivors can decide after a coordinator crash, but it assumes no network partitions and costs an extra round.

### 1.2 What is consensus?

**Consensus** = a group of processes must **agree on one value** (e.g. "who is the leader", "what is the next log entry") even though some may crash and messages may be delayed. A correct algorithm guarantees:

- **Agreement**: all correct processes decide the same value.
- **Validity**: the decided value was proposed by some process.
- **Termination**: every correct process eventually decides.

| Atomic commit (2PC) | Consensus (Paxos/Raft) |
|---|---|
| Needs **every** participant to vote YES; one NO or one silent node forces ABORT | Needs only a **majority (quorum)**; tolerates f crashed nodes out of 2f + 1 |
| Blocks if the coordinator fails at the wrong time | Keeps making progress while a majority is up and can talk |
| Decides commit/abort for one transaction | Agrees on any value, repeatedly (a replicated log) |

**Why consensus is harder:** in an **asynchronous** system (no bound on message delay, Lesson 1) you cannot tell a crashed node from a slow one. The famous **FLP result** (Fischer, Lynch, Paterson, 1985) proves that no deterministic

algorithm can *guarantee* consensus in a fully asynchronous system if even one process may crash. Practical algorithms (Paxos, Raft) therefore always keep **safety** (never two different decisions) and achieve **liveness** (actually deciding) using timeouts and leader election when the network behaves well enough. Commit is "easier" because it is allowed to give up (abort) or to block; consensus must make all correct nodes agree on a value without blocking on minority failures.

### 1.3 Paxos (in brief)

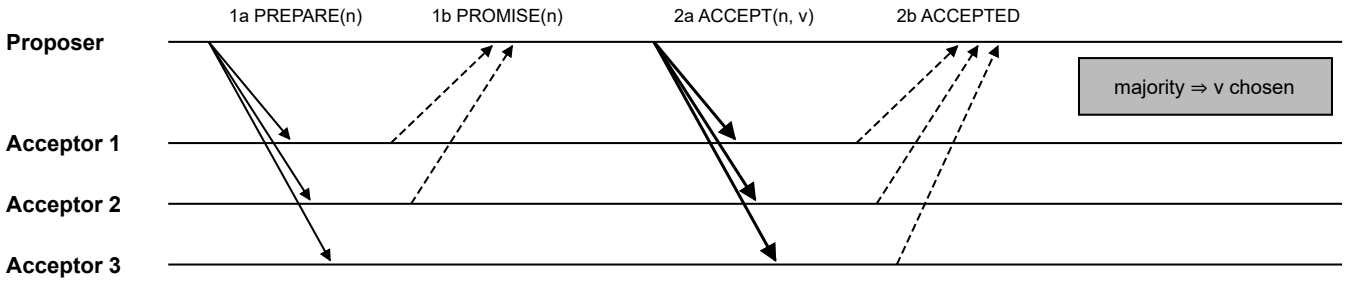



Figure 1 — Basic Paxos: two phases, each needs replies from a majority (here 2 of 3 is enough).

1. **Prepare:** a proposer picks a unique, increasing proposal number n and sends PREPARE(n). An acceptor **promises** not to accept any proposal numbered below n and reports any value it already accepted.
2. **Accept:** with promises from a majority, the proposer sends ACCEPT(n, v), where v is the value of the highest-numbered proposal already accepted (or its own value if none). Acceptors accept unless they promised a higher n. When a majority accepts, v is **chosen**.

Because any two majorities overlap in at least one acceptor, a later proposer always learns an already-chosen value and re-proposes it, so two different values can never be chosen. Paxos is correct but famously hard to understand and implement.

### 1.4 Raft

**Raft** (2014) solves the same problem but was designed to be understandable. It splits consensus into **leader election**, **log replication** and **safety**. Each node is a **follower**, **candidate** or **leader**. Time is divided into numbered **terms**; each term has at most one leader.

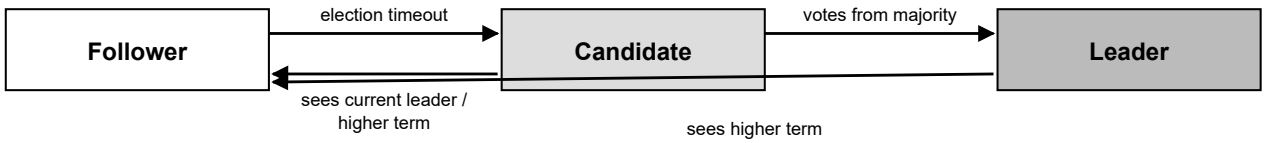



Figure 2 — Raft server states.

| Step (worked example: 5 servers S1–S5, leader S1 in term 1) | What happens |
|---|---|
| 1. Normal operation | S1 sends periodic **heartbeats** (empty AppendEntries) to all followers. |
| 2. S1 crashes | Followers stop receiving heartbeats. Each has a **random election timeout** (e.g. 150–300 ms) so they rarely time out together. |
| 3. S3 times out first | S3 becomes candidate, increments term to **2**, votes for itself, sends RequestVote to S2, S4, S5. |
| 4. Voting | Each server votes for at most one candidate per term, and only if the candidate's log is at least as up to date as its own. S2 and S4 vote for S3. |
| 5. Majority | S3 has 3 of 5 votes (itself + S2 + S4) ≥ majority (3) ⇒ **leader for term 2**; it starts sending heartbeats. S5 sees term 2 and stays follower. |
| 6. Log replication | A client sends a command to S3; S3 appends it to its log, sends AppendEntries; once a **majority** stores it, the entry is **committed** and applied to every state machine in the same order. |
| 7. S1 recovers | S1 still thinks it is leader of term 1, sees term 2 > 1, steps down to follower and its log is brought in line with S3's. |

With 5 servers Raft tolerates 2 failures (needs 3 alive). If two candidates split the votes, nobody wins, the term times out, and a new election with fresh random timeouts follows. **Used in:** etcd (the Kubernetes cluster database, Lesson 6), Consul; ZooKeeper uses the similar Zab protocol; Paxos variants are used in Google Chubby/Spanner. Distributed metadata (Ceph, HDFS Federation) needs such consensus (Lecture 07 slide 27).

**How consensus supports coordination in multi-agent systems:** agents use it to agree on a leader/coordinator, on the order of actions on a shared resource (a replicated log of "who holds the lock"), or on a shared plan, so that all agents act on the same decision even when some agents fail.

## 2. Multi-agent systems (MAS)

An **agent** is an autonomous software (or robotic) entity that **perceives** its environment through sensors and **acts** on it to achieve goals. A **multi-agent system** is a set of interacting agents, often on different machines — a distributed system where the components make their own decisions.

| Characteristic | Meaning |
|---|---|
| **Autonomy** | Acts without direct human or central control; has its own state and goals |
| **Reactivity** | Perceives and responds in time to changes in the environment |
| **Pro-activeness** | Takes initiative to pursue goals, not only reacting |
| **Social ability** | Communicates, cooperates and negotiates with other agents |
| In a MAS | Each agent has only a **local, partial view**; there is **no global control**; data is decentralized; computation is asynchronous — the same challenges as any distributed system (Lesson 1) |

**Agent types** (Lecture 01 outline): **reactive** (simple condition–action rules, fast, no internal model, e.g. a thermostat or obstacle-avoiding robot); **deliberative** (keeps a symbolic model of the world and plans, e.g. the **BDI** model: Beliefs, Desires, Intentions); **hybrid** (a fast reactive layer for emergencies plus a deliberative layer for planning, e.g. an autonomous car).

### 2.1 Communication

Agents exchange messages in an **Agent Communication Language (ACL)**. The **FIPA** standard (Foundation for Intelligent Physical Agents) defines FIPA-ACL: each message has a **performative** (the type of speech act: `inform`, `request`, `propose`, `accept-proposal`, `reject-proposal`, `cfp` = call for proposals), plus sender, receiver, content, a content language and an ontology (shared vocabulary). FIPA also defines platform services: an Agent Management System and a Directory Facilitator ("yellow pages"). JADE is a common FIPA-compliant platform.

### 2.2 Coordination and negotiation

- **Contract Net Protocol**: a manager broadcasts a **call for proposals** for a task; contractors reply with **bids**; the manager **awards** the task to the best bid; the winner reports results. (Like a tender.)
- **Auctions** (English, Dutch, sealed-bid) to allocate scarce resources.
- **Negotiation and argumentation**: exchange offers/counter-offers until agreement.
- **Organisational structures**: hierarchies, teams, roles; **blackboard** systems where agents share a common workspace.
- **Consensus / voting** (section 1) for agreeing on a leader or shared decision.

### 2.3 Resolving conflicts over shared resources

| Technique | Example |
|---|---|
| **Mutual exclusion / locking** (central lock manager, token passing, or a consensus-backed lock) | Only the agent holding the token may use the charging station |
| **Priorities and timestamps** (older/higher-priority request wins) | Ambulance agent overrides normal traffic agents |
| **Negotiation / auctions** | Delivery drones bid for a landing slot |
| **Optimistic vs pessimistic concurrency** | Pessimistic: lock before use. Optimistic: act, then check for conflict and retry (better when conflicts are rare) |
| **Social laws / conventions** | "Keep left" rule avoids collisions without messages |

Applications: autonomous vehicles, IoT and smart grids, robot warehouses, supply chains, traffic control.

## 3. Distributed search engines

### 3.1 Pipeline

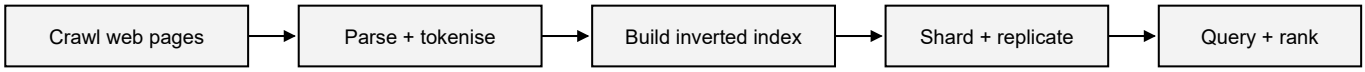



Figure 3 — Search engine pipeline; crawling and index building are big batch jobs (MapReduce, section 5).

**Crawling**: many crawler machines download pages, follow links, and revisit pages to keep the index fresh. **Challenges** (Lecture 01 outline): **freshness** (pages change constantly) and **coverage** (the web is huge; you cannot crawl it all).

### 3.2 Inverted index (worked example)

An **inverted index** maps each **term** (word) to a **posting list**: the documents containing it (plus positions/frequencies). It is "inverted" because instead of document → words, it stores word → documents, so a query only reads the lists of its words.

| Documents | Inverted index |
|---|---|
| D1: "cloud storage systems"<br>D2: "distributed storage"<br>D3: "cloud computing" | cloud → D1, D3<br>storage → D1, D2<br>systems → D1<br>distributed → D2<br>computing → D3 |
| Query "cloud AND storage": intersect {D1, D3} ∩ {D1, D2} = **{D1}**. Query "cloud OR storage" = {D1, D2, D3}. | |

### 3.3 Sharding and query processing

The index is far too big for one machine, so it is split into **shards** (partitions), each replicated for fault tolerance and throughput.

| | Document-partitioned (local index) | Term-partitioned (global index) |
|---|---|---|
| Split | Each shard holds a full index for a subset of documents | Each shard holds the complete posting lists for a subset of terms |
| Query | Sent to **all** shards in parallel; each returns its top results; merged | Sent only to the shards holding the query's terms; lists are combined |
| Pros/cons | Simple, balanced, easy to add documents; every query touches every shard | Fewer shards per query; hot terms create hot spots; big lists move across the network |

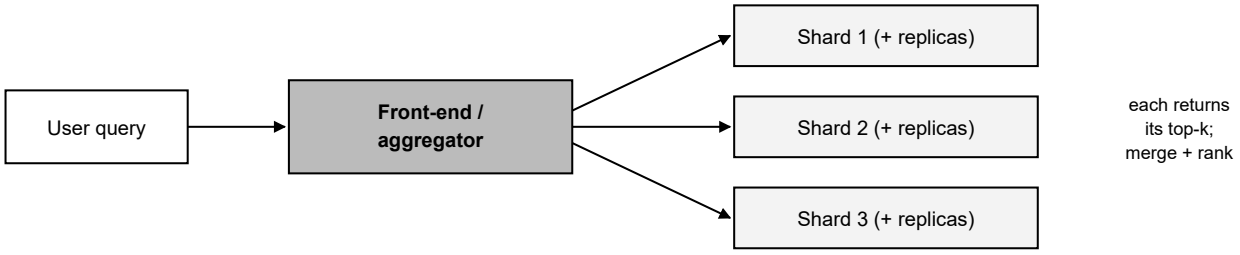



Figure 4 — Scatter-gather query over a document-partitioned index.

### 3.4 Ranking: PageRank (worked example)

PageRank scores a page by how many important pages link to it. With damping factor d = 0.85 and N pages: **PR(p) = (1 − d)/N + d × Σ PR(q)/out(q)** over pages q linking to p (out(q) = number of links leaving q).

Links: A → B, A → C, B → C, C → A. N = 3, start all at 1/3, (1 − d)/N = 0.05.

| Page | Iteration 1 | Converged (after many iterations) |
|---|---|---|
| A | 0.05 + 0.85 × (C/1) = 0.05 + 0.85 × 0.333 = 0.333 | ≈ 0.388 |
| B | 0.05 + 0.85 × (A/2) = 0.05 + 0.85 × 0.167 = 0.192 | ≈ 0.215 |
| C | 0.05 + 0.85 × (A/2 + B/1) = 0.05 + 0.85 × 0.5 = 0.475 | ≈ 0.397 |

C ranks highest (two incoming links); scores sum to 1. On the web this is computed over billions of pages as repeated MapReduce jobs.

### 3.5 Centralized vs distributed/P2P search

| | Centralized index | Distributed (sharded cluster, e.g. Google, Elasticsearch) | P2P search (e.g. Gnutella, DHT-based) |
|---|---|---|---|
| Scalability | Limited by one machine | High: add shards/replicas | Grows with peers, but flooding/lookup cost grows too |
| Fault tolerance | Single point of failure | Replicas take over; partial results if a shard is down | No central SPOF, but peers churn; results incomplete |
| Query latency | Low at small scale | Low: parallel scatter-gather in one data centre | Higher and variable (many hops over the Internet) |
| Control/freshness | Easy | Managed by one operator | Hard: no one controls the whole index |

## 4. Peer-to-peer systems

### 4.1 Unstructured P2P (Gnutella-style flooding)

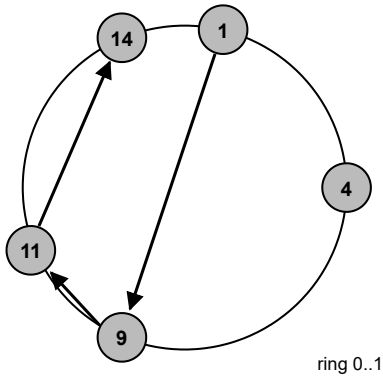
Peers connect to random neighbours; there is no rule about where data is stored. To search, a peer sends the query to all neighbours, who forward it to theirs, up to a **TTL** (time-to-live, a hop limit).

**Worked example:** each peer has 3 new neighbours and TTL = 3: messages = 3 + 9 + 27 = **39** for one search, reaching at most 39 peers. TTL = 5 gives 3 + 9 + 27 + 81 + 243 = 363. Traffic grows exponentially, yet a rare file beyond the TTL is **not found** even if it exists.

### 4.2 Structured P2P: Distributed Hash Tables (DHT)

A **DHT** gives every node and every key an ID from the same space (by hashing, e.g. SHA-1) and has a rule for which node stores which key, so a lookup can be routed to exactly the right node in about **O(log N)** hops.

**Chord**: IDs on a ring of size $2^m$. Key k is stored at its **successor**: the first node whose ID ≥ k (clockwise). Each node keeps a **finger table** of m entries: finger i = successor(n + $2^{i-1}$). Lookups jump about half the remaining distance each hop.




**Finger table of node 1**

i = 1: succ(1 + 1 = 2) = 4
i = 2: succ(1 + 2 = 3) = 4
i = 3: succ(1 + 4 = 5) = 9
i = 4: succ(1 + 8 = 9) = 9

**Lookup key 12 from node 1:**
node 1: closest finger before 12 = 9 → go to 9
node 9: fingers 11, 11, 14, 1 → closest before 12 = 11
node 11: successor 14 and 12 ∈ (11, 14]
⇒ **key 12 is stored at node 14 (path 1 → 9 → 11 → 14)**

Figure 5 — Chord worked example: nodes 1, 4, 9, 11, 14 on a 16-ID ring.

**Kademlia**: the distance between two IDs is their bitwise **XOR**; each node keeps "k-buckets" of contacts at distances $2^i$, and queries several of the closest nodes in parallel. XOR is symmetric, so normal traffic keeps routing tables fresh; it handles churn well. Used by BitTorrent's DHT and IPFS.

**BitTorrent** (content distribution): a file is split into pieces; a tracker or the DHT finds peers in the "swarm"; peers download different pieces from many peers at once and upload pieces they have ("tit-for-tat" rewards uploaders); rarest pieces are fetched first. The more popular the file, the faster it spreads.

### 4.3 Structured vs unstructured

| Trade-off | Unstructured (Gnutella flooding) | Structured (Chord, Kademlia DHT) |
|---|---|---|
| Lookup efficiency | Poor: flooding costs many messages; not guaranteed (TTL); good for keyword/partial searches and popular items | Efficient and guaranteed: O(log N) hops for an exact key; no keyword search by itself |
| Churn tolerance (peers joining/leaving) | High: no structure to repair; a leaving peer just disappears | Lower: finger tables and key ownership must be repaired on joins/leaves (stabilisation); Kademlia copes better than Chord |
| Implementation complexity | Simple | More complex (hashing, routing tables, stabilisation, replication of keys) |
| Load / placement | Data anywhere | Data placed by hash: balanced, but no control over where data lives |

**Other outline topics (Lecture 01 slide 11): reputation and trust management** (rating peers to avoid free-riders and malicious peers), legal and ethical issues (copyright), and **blockchain**: a distributed ledger replicated across peers, agreed by consensus — **Proof of Work** (miners solve a costly puzzle), **Proof of Stake** (validators chosen by the stake they lock), **PBFT** (Practical Byzantine Fault Tolerance: votes among known nodes, tolerates f Byzantine of 3f + 1; used in permissioned chains such as Hyperledger); smart contracts (Ethereum) run code on the ledger.

## 5. MapReduce and cloud programming models

### 5.1 MapReduce

A programming model (Google, 2004) for processing huge datasets on clusters of commodity machines. The programmer writes only two functions; the framework handles distribution, scheduling and failures.

- **map(key, value) → list of (k2, v2)**: run in parallel on each input split (e.g. each 64/128 MB block of a GFS/HDFS file, processed on the machine that stores it: **data locality**).

- **Shuffle & sort**: the framework groups all values with the same k2 and sends them to one reducer.
- **reduce(k2, list of v2) → output**: combines each group.
- **Fault tolerance**: a master pings workers; tasks of a failed worker are re-executed elsewhere; slow "stragglers" get backup copies.

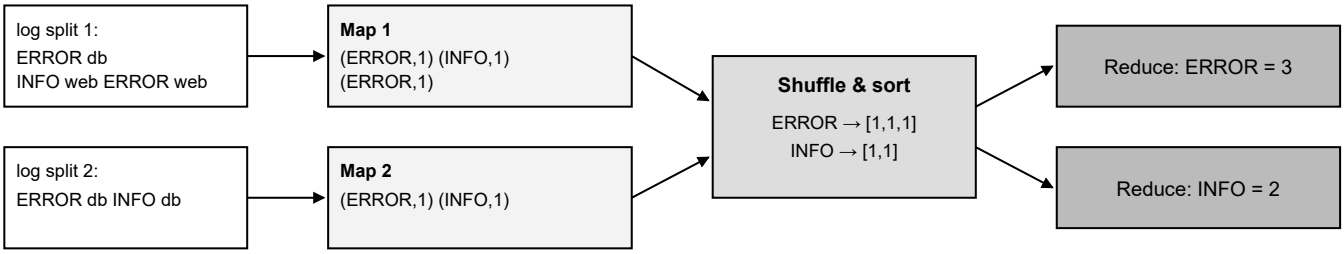



Figure 6 — MapReduce log analysis: count log lines per level (map emits (level, 1) per line).

```
# Part C-style word count (Python, simulated workers)
from collections import defaultdict
from multiprocessing import Pool

def map_fn(text):                       # one split -> list of (word, 1)
    return [(w.lower(), 1) for w in text.split()]

def reduce_fn(item):                    # (word, [1,1,..]) -> (word, count)
    word, ones = item
    return word, sum(ones)

if __name__ == "__main__":
    splits = ["cloud data cloud", "data node"]
    with Pool(2) as p:
        mapped = p.map(map_fn, splits)  # map phase in parallel processes
    groups = defaultdict(list)          # shuffle: group by key
    for pairs in mapped:
        for w, one in pairs:
            groups[w].append(one)
    with Pool(2) as p:
        print(dict(p.map(reduce_fn, groups.items())))   # {'cloud': 2, 'data': 2, 'node': 1}
```

**How it would scale for real:** input in HDFS/S3 split into blocks; thousands of map tasks run where the blocks are; shuffle over the network; results written back to the DFS. **Limits of a local simulation:** one machine's CPU, RAM and disk; no real network shuffle cost; no machine failures, stragglers or data locality to handle.

### 5.2 Cloud programming models compared

| Model | Idea | Suits | Examples |
|---|---|---|---|
| **MapReduce / batch dataflow** | map → shuffle → reduce over huge static datasets | Log analysis, indexing, ETL; high throughput, not low latency | Hadoop MapReduce, Spark |
| **Serverless / FaaS** | Small stateless functions triggered by events; scale 0 → many; pay per call (Lesson 2) | Event-driven, bursty, short tasks (resize an uploaded image) | AWS Lambda, Azure Functions |
| **Actor model** | Independent "actors" with private state communicate only by asynchronous messages; each handles one message at a time | Highly concurrent, stateful, interactive systems (chat, games, IoT) | Akka, Erlang/Elixir, Orleans |
| Stream processing | Continuous computation on unbounded event streams | Real-time analytics, fraud detection | Kafka Streams, Flink |

## 6. Related outline topics: ACID vs BASE, concurrency control

| ACID (classic transactions) | BASE (many NoSQL / AP systems) |
|---|---|
| **A**tomicity (all or nothing), **C**onsistency (rules kept), **I**solation (concurrent transactions don't interfere), **D**urability (committed data survives crashes) | **B**asically **A**vailable, **S**oft state (state may change without input as replicas converge), **E**ventually consistent |
| Fits CP choices: banking (Lesson 1) | Fits AP choices: social feeds, shopping carts |

**Pessimistic concurrency**: lock data before using it (two-phase locking); safe under heavy contention but can block and cause **distributed deadlock** (T1 waits for a lock held by T2 on another server, and T2 waits for T1: detected by building a global wait-for graph and finding a cycle, or avoided with timeouts). **Optimistic concurrency**: work without locks, then validate at commit time and abort/retry if someone else changed the data; best when conflicts are rare.

## 7. Exam-style questions with model answers

**Q1. Explain why consensus is harder than transaction commitment. (5 marks)**
Atomic commit only needs a unanimous yes/no on one transaction and may abort or block when something fails; any NO or silent node aborts it. Consensus must make all correct processes agree on a value while tolerating crashed processes, needing only a majority, and must keep making progress. In an asynchronous system a crashed process cannot be distinguished from a slow one, and the FLP result shows no deterministic algorithm can guarantee consensus there with even one crash; practical protocols (Paxos, Raft) guarantee safety always and liveness using timeouts and leader election.

**Q2. Describe Raft leader election with an example. (6)**
Servers are followers, candidates or leaders; time is divided into terms. The leader sends heartbeats. If a follower's randomised election timeout expires without a heartbeat, it becomes a candidate, increments the term, votes for itself and requests votes; each server grants one vote per term to a candidate whose log is at least as up to date. A candidate with a majority (e.g. 3 of 5) becomes leader and sends heartbeats; a server that sees a higher term steps down. Split votes cause a new election after another random timeout.

**Q3. What are the key characteristics of a multi-agent system? How do agents coordinate and resolve conflicts over shared resources? (8)**
Agents are autonomous, reactive, proactive and social; in a MAS each has a partial local view, there is no global control, and data and computation are decentralized and asynchronous. They communicate with an agent communication language such as FIPA-ACL (performatives like inform, request, propose). They coordinate through protocols like the Contract Net (call for proposals, bids, award), auctions, negotiation, organisational roles or consensus. Conflicts over shared resources are resolved with mutual exclusion/locking or token passing, priorities and timestamps, negotiation or auctions, optimistic or pessimistic concurrency control, and social conventions.

**Q4. Explain how a distributed search engine indexes and retrieves data using inverted indexes and sharding. (8)**
Crawlers fetch pages; pages are tokenised and an inverted index maps each term to a posting list of documents. The index is split into shards (by document or by term) spread over many nodes and replicated. A query goes to a front-end that scatters it to the relevant shards in parallel (all shards for a document-partitioned index); each shard looks up the posting lists, intersects/scores them and returns its top results; the front-end merges and ranks them (e.g. with PageRank and relevance) and returns the answer. Replicas give fault tolerance and throughput.

**Q5. Compare centralized and distributed/P2P search architectures. (6)**
Centralized: simple and fast at small scale but limited by one machine and a single point of failure. Distributed cluster: scales by adding shards and replicas, tolerates node failures, and keeps latency low with parallel scatter-gather. P2P: no central server and scales with peers, but lookups take many hops or floods, latency is higher and variable, results may be incomplete, and churn and lack of control make freshness and quality hard.

**Q6. Compare structured and unstructured P2P systems. (8)**
Unstructured (Gnutella): peers link randomly and search by flooding with a TTL; simple and very tolerant of churn, supports keyword search, but lookups are expensive (messages grow exponentially) and not guaranteed to find rare items. Structured (Chord, Kademlia): a DHT hashes nodes and keys into one ID space and assigns each key to a

specific node; lookups route in O(log N) hops and always find an existing key, but the routing structures must be maintained when peers join and leave, which is more complex and less tolerant of heavy churn; only exact-key lookup is supported directly.

**Q7. In a Chord ring with m = 4 and nodes 1, 4, 9, 11, 14, which node stores key 12, and what is node 1's finger table? (5)**
Key 12 is stored at its successor, node 14. Node 1's fingers are successor(2) = 4, successor(3) = 4, successor(5) = 9, successor(9) = 9. Lookup path 1 → 9 → 11 → 14.

**Q8. Explain MapReduce with a word-count example. How does it tolerate failures? (6)**
Input is split into blocks; map tasks run in parallel and emit (word, 1) for each word; the framework shuffles and groups pairs by word; reduce tasks sum each list (cloud = 2, data = 2, node = 1). The master monitors workers and re-executes tasks of failed workers on other machines, and starts backup copies of slow tasks; map tasks run near their data to reduce network use.

**Q9. Compare MapReduce, serverless/FaaS and the actor model. (6)**
MapReduce: batch processing of huge static datasets through map, shuffle and reduce; high throughput, high latency. Serverless: stateless functions triggered by events, auto-scaling to zero, pay per invocation; good for bursty short tasks; cold starts and no long-running state. Actor model: actors with private state exchanging asynchronous messages; good for highly concurrent, stateful, interactive systems.

**Q10. Distinguish ACID and BASE. (4)**
ACID: atomicity, consistency, isolation, durability; strong guarantees, suits banking (CP). BASE: basically available, soft state, eventually consistent; favours availability and scale, suits feeds and carts (AP).

## 8. Common mistakes

| Mistake | Correct idea |
|---|---|
| Consensus needs every node to agree before deciding | Paxos/Raft need only a majority; 2PC needs everyone. |
| Raft can have two leaders in the same term | At most one leader per term (one vote per server per term, majority needed). |
| Chord stores key k at the node before k | At its successor: the first node with ID ≥ k. |
| Flooding always finds the file | Only within the TTL; rare items may be missed. |
| DHTs support keyword search natively | They support exact-key lookup; keyword search needs extra indexing. |
| An inverted index maps documents to words | It maps words (terms) to documents (posting lists). |
| MapReduce is good for low-latency queries | It is a high-throughput batch model. |
| Reactive agents plan ahead | Reactive = condition–action rules; deliberative agents plan (BDI); hybrid combines them. |

## Cheat sheet

| Commit & consensus | |
|---|---|
| 2PC flaws | Blocking, coordinator SPOF, locks held, latency; 3PC fixes blocking (crash-only, no partitions) |
| Consensus | Agreement, validity, termination; majority quorum; f failures of 2f + 1 |
| Harder because | Async: crash vs slow indistinguishable; FLP; must not block on minority |
| Paxos | PREPARE(n) → PROMISE → ACCEPT(n, v) → ACCEPTED by majority |
| Raft | Follower / candidate / leader; terms; random timeouts; majority vote; heartbeats; log entry committed on majority |

| Multi-agent systems | |
|---|---|
| Traits | Autonomy, reactivity, pro-activeness, social ability; local view, no global control |
| Types | Reactive, deliberative (BDI), hybrid |
| Talk | FIPA-ACL performatives: inform, request, propose, cfp |
| Coordinate | Contract Net, auctions, negotiation, roles, blackboard, consensus |
| Conflicts | Locks/tokens, priorities, auctions, optimistic/pessimistic, conventions |

| ACID vs BASE | |
|---|---|
| ACID | Atomic, Consistent, Isolated, Durable (CP) |
| BASE | Basically Available, Soft state, Eventually consistent (AP) |

| Distributed search | |
|---|---|
| Pipeline | Crawl → parse → inverted index → shard + replicate → query + rank |
| Inverted index | Term → posting list; AND = intersect |
| Sharding | By document (query all shards) or by term (query term shards) |
| PageRank | PR = (1 − d)/N + d Σ PR(q)/out(q), d = 0.85 |
| Challenges | Freshness, coverage |

| | Unstructured | Structured DHT |
|---|---|---|
| Example | Gnutella | Chord, Kademlia |
| Lookup | Flood, TTL; not guaranteed | O(log N), guaranteed |
| Churn | Tolerant | Needs repair |
| Complexity | Simple | Complex |
| Search | Keyword | Exact key |

Chord: key at successor; finger i = succ(n + $2^{i-1}$).
Kademlia: XOR distance. BitTorrent: pieces, swarm, tit-for-tat

| Programming models | |
|---|---|
| MapReduce | Map → shuffle/sort → reduce; data locality; re-run failed tasks; batch |
| FaaS | Event-driven stateless functions; scale to 0; pay per call |
| Actor | Private state + async messages; concurrent, stateful |
| Blockchain | Replicated ledger; PoW, PoS, PBFT (3f + 1) |